The DMM6500 trigger model — a working manual

Ian Ameline · version 1.56 · MIT licence

The trigger model is not a trigger setting. It is a small programmable machine with its own instruction store, and the instrument's own scanning feature is written in it. This describes what it is, and what it does as opposed to what the reference manual says it does.

Claims marked **MEASURED** were established on this instrument — DMM6500, firmware **1.7.17a** — and each names its experiment. Everything else comes from the *DMM6500 Reference Manual*, DMM6500-901-01 Rev. A/April 2018, cited where the location is pinned down. Anything marked NOT ESTABLISHED has neither a measurement nor a citation and should be read as an open question.

---

1. The machine

program store	**63 blocks**, indexed from 1 (§9-49)
sequences	concurrent, each `START` … `END`; the event log calls them **paths**
control flow	8 branch types — always, once, once-excluded, on-event, counter, delta, limit-constant, limit-dynamic — plus `RESET_BRANCH_COUNT`, which resets a counter and does not branch
signals	8 notify blocks, 2 blenders, 4 timers, 6 digital I/O lines, 8 LAN objects, 3 TSP-Link lines
state	`trigger.model.state()` → `EMPTY`, `BUILDING`, `RUNNING`, `WAITING`, `IDLE`, `ABORTED`, `FAILED`, `PAUSED` observed; `ABORTING` never seen in **200 samples taken with nothing between them and `abort()`**, so the transition is atomic at this granularity (MEASURED, P6)
state's return	**the first value is `userdata`, not a string** — it prints as `trigger.STATE_WAITING` and concatenates like one, then raises `-286 string expected, got userdata` the moment anything treats it as one. Compare with `==` against `trigger.STATE_*`. The arity is not fixed: an `EMPTY` model returned exactly three, `(state, state, 3)`, and a `FAILED` one printed five, `FAILED FAILED 1 FAILED 0` (MEASURED, same)

A program counter, conditional branches, per-block loop counters, and inter-sequence signalling. No general storage and no arithmetic, so it is not a computer — but a rendezvous protocol between two sequences is expressible, and that is what scanning uses.

2. The instruction set

```
BUFFER_CLEAR          [buffer]
MEASURE_DIGITIZE      [buffer [, count]]        count: a value, COUNT_INFINITE,
COUNT_STOP, COUNT_AUTO
WAIT                  event [, clear [, logic, event [, event]]]
DELAY_CONSTANT        seconds
DELAY_DYNAMIC         delayList, index
NOTIFY                trigger.EVENT_NOTIFYN
NOP
LOG_EVENT             eventNumber, message
DIGITAL_IO            bitPattern, bitMask
CONFIG_RECALL         "list" [, index]
CONFIG_NEXT           "list"
CONFIG_PREV           "list"
RESET_BRANCH_COUNT    counter
BRANCH_ALWAYS         block
BRANCH_ONCE           block
BRANCH_ONCE_EXCLUDED  block
BRANCH_ON_EVENT       event, block
BRANCH_COUNTER        targetCount, block
BRANCH_DELTA          targetDifference, block, measureBlock
BRANCH_LIMIT_CONSTANT limitType, limitA, limitB, block [, measureBlock]
BRANCH_LIMIT_DYNAMIC  limitType, limitNumber, block [, measureBlock]
```

Three more exist and are **undocumented** — `trigger.BLOCK_START`, `trigger.BLOCK_END` and `trigger.BLOCK_CHANNEL_ACTION` are real TSP constants (MEASURED, E3c) and appear in no `setblock` usage list. A fourth, `WRITE_USB`, is emitted by `scan.export()` and the string appears **zero** times in the reference manual (MEASURED, §9 worked example below).

3. Events

The signal namespace a `WAIT` or a `BRANCH_ON_EVENT` can name:

```
EVENT_ANALOGTRIGGER   the signal on INPUT HI/LO crossing a comparator level
EVENT_DISPLAY         the front-panel TRIGGER key
EVENT_EXTERNAL        rear EXT TRIG IN
EVENT_COMMAND         a bus trigger (*TRG, GPIB GET, VXI-11 device_trigger)
EVENT_TIMERN          trigger timer N expired            (1..4)
EVENT_NOTIFYN         notify block N executed            (1..8)
EVENT_BLENDERN        blender N fired                    (1..2)
EVENT_DIGION          edge on digital input N            (1..6)
EVENT_LANN            LXI trigger packet on object N      (1..8)
EVENT_TSPLINKN        edge on TSP-Link sync line N        (1..3)
EVENT_SCAN_CHANNEL_READY / EVENT_SCAN_MEASURE_COMPLETE / EVENT_SCAN_COMPLETE /
EVENT_SCAN_ALARM_LIMIT
EVENT_NONE            no event — illegal as a WAIT's first event
```

Digital I/O, GPIB and TSP-Link need a communications accessory card (KTTI-GPIB, KTTI-TSP, KTTI-RS232).

4. Writing a program

```
trigger.model.load("Empty")                  -- empties the store; any load() replaces
it
trigger.model.setblock(1, trigger.BLOCK_BUFFER_CLEAR, buf)
trigger.model.setblock(2, trigger.BLOCK_WAIT, trigger.EVENT_ANALOGTRIGGER,
                       trigger.CLEAR_ENTER, trigger.WAIT_OR, trigger.EVENT_DISPLAY)
trigger.model.setblock(3, trigger.BLOCK_MEASURE_DIGITIZE, buf, 20000)
trigger.model.initiate()                     -- returns IMMEDIATELY; the model runs in
firmware
print(trigger.model.getblocklist())          -- disassembles the store
```

`setblock` does not clear the store (MEASURED, E1d). It overwrites the blocks you name and leaves the rest in place. Writing a 3-block program over a previous 5-block one leaves blocks 4 and 5 live: in E1d a leftover `BRANCH_COUNTER 1000 → 2` turned a straight-line program into a thousand-iteration loop over the new `WAIT`. **Always `load("Empty")` first.** The name is case-insensitive.

Any `load()` replaces the store wholesale, and truncates it (MEASURED, P4, against eight `NOP`s):

```
load("SimpleLoop", …)      5-block template   -> 5 blocks, blocks 6-8 GONE
load("LoopUntilEvent", …)  6-block template   -> 6 blocks, blocks 7-8 GONE
load("Empty")                                 -> 0 blocks
trigger.model.abort()                         -> 8 blocks, untouched
trigger.clear()                               -> 8 blocks, untouched
setblock(1, BLOCK_NOP)                        -> 8 blocks, untouched
```

A short template over a longer program leaves no tail blocks, so there is no stale-tail trap. The accurate rule: **any `load()` replaces the store entirely; `abort()`, `trigger.clear()` and `setblock` do not clear it; `load("Empty")` is the only way to leave it empty.** Note `trigger.clear()` exists and clears trigger EVENT detectors, not the program — the name invites exactly the wrong assumption.

`initiate()` is overlapped. It arms the machine and returns; Lua keeps running, and reading the buffer while the model runs works — MEASURED, §7.

The measure function cannot be changed while the model runs, and the refusal is out of band (MEASURED, P2f). With a model genuinely held in `STATE_WAITING`, `dmm.digitize.func = …` returned `pcall` true, left the function unchanged, left the state `WAITING`, and posted **2727, "Command prohibited while trigger model is running"**. Writing the value it already held posted 2727 too, so the refusal is on the write, not on the value.

NOT ESTABLISHED: that switching between local and remote control aborts a running model. It is not scriptable from the socket, and the local/remote text at 15-2 says only that the front panel is disabled and that you are prompted to switch control. The experiment is written and takes one press — P7 parks a model on `EVENT_EXTERNAL` with no marker and polls from inside for 40 s, so a press that aborts the model is visible as the second the state leaves `RUNNING` and a press that does not is visible as a model still parked at 40.000 s. **What it cannot do is tell a model that ignored the press from a window in which nobody pressed**, so it reports only what it saw and this claim stays open until a press is confirmed.

5. Neither `setblock` nor `initiate` reports failure

This is the most important thing here, and it is the opposite of what a Lua programmer expects.

call	`pcall` says	what happened
`setblock(64, …)`	**true**	refused — event **1130**, "Parameter block number, expected value from 1 to 63" (MEASURED, E3a)
`setblock(1, BLOCK_WAIT, EVENT_NONE)` then `initiate()`	**true**, **true**	state → **`STATE_FAILED`**, event **2707**, "Block number 1 has an invalid event ID in parameter 1" (MEASURED, E3b)

| `dmm.digitize.func = …` while the model runs | **true** | discarded — event **2727**, "Command prohibited while trigger model is running" (MEASURED, P2f) |

| `trigger.model.pause()` on a model that is not running | **true** | refused — event **2767**, "Trigger model cannot be paused unless it is already running" (MEASURED, P4) | | `trigger.model.pause()` on a model that **is** running | **true** | honoured, and this is the one call here that did what it said: the state became `trigger.STATE_PAUSED` and `resume()` put it back to `RUNNING` (MEASURED, P6) |

The whole trigger subsystem reports refusals out of band, not just these two calls: a block write, an ordinary attribute write and a plain function call each return success and file the reason as an event, with its own number. Treat a successful return from anything in `trigger.` as carrying no information about whether the work happened.

A `pcall` verdict on any of them is a constant. **`trigger.model.state()` is the only verdict that means anything**, and the reason lives in `eventlog.next()`. A program that builds and initiates "successfully" can be dead. `setblock(63, …)` is accepted with no event, so the ceiling is exactly as documented — enforced, but reported out of band.

Nothing in §9 or §15 says this, and no example in the manual checks for it.

A command arriving on the remote interface ABORTS a running model

A model that is waiting cannot be WATCHED from a host. Every command that arrives on the socket while one is initiated aborts it, and the abort is filed as event **2728**, "Trigger model path #1 has been aborted" (MEASURED, P5). The control is what makes it conclusive — one chunk that initiates, delays four seconds **inside the instrument**, and reports:

	state	readings	events
at `initiate()`	`STATE_RUNNING`	9	—
4 s later, nothing sent in between	**`STATE_WAITING`**	21 100	1, which is 2731 "initiated"
the same program, state polled from the host twice a second	`STATE_RUNNING`, then **`STATE_ABORTED`** within 0.5 s	frozen at a **variable** count — 4 500, 9 233, 10 645, 11 756, 14 159 over five runs	2731, then **2728**

The variable count is the tell: the model dies when the next command arrives, not at a point of its own. Four program shapes behave identically — `LoopUntilEvent` at position 5 and at position 0, and the hand-built three- and two-block forms — so this is the interface and not the program.

So a wait has to be polled from inside the instrument, in the same Lua call that initiated it, with `delay()` between looks. That is not a style preference; it is the only way the model survives.

And the state at `initiate()` is not the state of the wait. `STATE_RUNNING` with nine readings already taken is the model executing the blocks AHEAD of its wait — a buffer clear, a zero delay and the start of an infinite digitize. Code that reads "not waiting" as "the event has arrived" is therefore wrong on its first look.

It is not "a millisecond later". That figure was inferred from the nine readings and never measured. `STATE_WAITING` arrives about **0.19 s** after `initiate()`, and §8 has the distribution, what drives it, and the defect it caused.

And Keithley's own host code relies on the opposite, which is not yet reconciled. `Instrument_Examples/DMM6500/Streaming_Examples/00_Stream_Data_from_DMM6500/Stream_DMM6500.py` queries an initiated model over a **raw socket on port 5025**, in a loop, for the whole length of a run, and depends on the model continuing: its `get_data()` blocks inside the instrument until the buffer grows, so an abort on the first query would deadlock the host. It never calls `trigger.model.abort()` at all.

It does not work here, and the rule is wider than the table above suggests. A single `print(defbuffer1.n)` from the host aborts a model **in a measure block** — `RUNNING` at n = 2 going to `ABORTED` at n = 140 — and one **parked in a `BLOCK_WAIT` on `EVENT_ANALOGTRIGGER` that cannot fire**, with the comparator at 9.9 V against a measured ±10 mV idle so the wait was genuine: `RUNNING` at n = 12 going to `ABORTED` at n = 200. Both logs hold `2731` then `2728`. So the block type is not the variable, there is no surviving case, and Keithley's three streaming examples cannot work as written on this instrument.

But the arriving command may not be the mechanism, and that is now open. A model initiated in one chunk, with no print in the chunk and no traffic from that socket afterwards, died anyway: three repeats at **427.6, 436.1 and 441.4 ms** — 17 654, 17 442 and 17 104 readings at 40 kS/s — and two earlier runs under different command sequences at 5.4 ms and 246.3 ms. Polled from **inside** one chunk the same model reached 39 400 readings. So something periodic ends it, and a loaded application's own panel tick — 2 Hz in the case measured — is the named suspect.

The confound cannot be removed from that run: a second client held a socket to the instrument throughout, so "no traffic" means none from the measuring socket. It also reframes the variability the original finding leaned on — 4 500 to 14 159 readings was read as "it dies when the next command arrives", and a model that dies on its own clock would look the same. **The operational rule is unchanged either way**: poll a wait from inside the instrument, because every host-side attempt to watch one *with an application loaded* has failed.

The confound is removable after all, and it exonerates the readback (MEASURED, P6, on an instrument minutes out of a power cycle with **no application loaded** — its globals absent — and one socket open). A model parked on `EVENT_EXTERNAL` with no marker arriving answered **2440 consecutive host state queries over 4.001 s** and was still in `STATE_WAITING` at the last of them — an order of magnitude past the 427–441 ms the three deaths were measured at, and ~2 400 more queries than the one it would take if asking were enough. **So a host query does not kill a parked wait. What killed those three arrived with the loaded application**, which leaves its panel tick as the only suspect still standing rather than one of two. Poll from inside because of what else is running, not because of the question.

And the first query is still answered truthfully, which is what makes this so expensive to diagnose. The abort lands *behind* the reply and the model runs on for about 140 ms, so a host-side probe gets a plausible number back and only the next read shows the damage. A first attempt to measure the front-panel key this way returned three arms that all looked like a disabled key; each had been killed by its own readback.

One caution on reading those vendor scripts as evidence at all: of the three, one (`KEIDMM6500_Stream_Measured_Dual_Meter.py`) talks **USBTMC** through PyVISA rather than a socket, and pairs a DMM6500 with a DAQ6510 — so it says nothing about this interface. `Stream_DMM6500.py` is the only raw-socket DMM6500 case, and it is the one the measurement above refutes.

6. `BLOCK_WAIT` : three events, and a latch

```
setblock(n, trigger.BLOCK_WAIT, event [, clear [, logic, event [, event]]])
```

`logic` is `trigger.WAIT_OR` or `trigger.WAIT_AND` and governs the whole list — you cannot mix them in one block (§15-413). Three events in one block means **a blender is unnecessary for a simple OR**: "the analog trigger, OR the front key, OR the rear BNC" is one block (MEASURED, E1c).

The wait latches. An event that fires before execution reaches the block is remembered, and `clear` decides what happens:

`clear`	an event that already fired	MEASURED (E3g)
`trigger.CLEAR_NEVER` *(default)*	passes straight through	state `IDLE`, readings taken
`trigger.CLEAR_ENTER`	discarded; waits for a fresh one	state **`WAITING`**, **0 readings**

The default is the dangerous one. A capture armed on the TRIGGER key with `CLEAR_NEVER` fires on a press made minutes earlier. **Use `CLEAR_ENTER` for anything a human can trigger.** Use `CLEAR_NEVER` deliberately when you want a semaphore — see §9. The latch also clears at the start block and on exiting the wait, and a `BRANCH_ON_EVENT` consumes it too, which is what makes a silence detector possible — see §13.

The default `logic` is `AND`, not `OR`. A wait built as `(event, clear)` with no logic argument disassembles as `LOGIC: AND`, while `LoopUntilEvent` emits `LOGIC: OR`. Harmless for one event, and a trap the moment a second is added.

7. Pre-trigger is a program shape, not a setting

Digitize infinitely into a circular buffer, wait, then digitize a counted burst. The counted block stops the infinite one.

```
1) BUFFER_CLEAR      buf
2) MEASURE_DIGITIZE  buf, trigger.COUNT_INFINITE   -- fills continuously, wraps
3) WAIT              <event>, CLEAR_ENTER
4) MEASURE_DIGITIZE  buf, <post-trigger count>     -- stops the infinite block
```

MEASURED (E3d). 1 kSa/s into a 5000-reading buffer, a timer firing the wait at 2 s, post-trigger count 500: `buf.n` read 503 at ~0.5 s, 1505 at ~1.5 s, **2501** final — 2001 pre-trigger readings plus 500 post. The buffer accumulates *across* the wait.

The pre-roll is not optional: an analog-trigger wait cannot be the first block (MEASURED, P3). Drop the `COUNT_INFINITE` block and the program becomes

```
1) WAIT              EVENT_ANALOGTRIGGER, CLEAR_ENTER
2) MEASURE_DIGITIZE  buf, 400
```

which waits for ever. Three seconds on a live 1 kHz square wave, comparator armed dead centre of its 0-3 V swing, gave `STATE_WAITING` with **zero readings**, no event and no error. The control is decisive: the same level, the same slope, the same signal and the same session fired in **under 0.01 s** once a `COUNT_INFINITE` block was put ahead of the wait.

`EVENT_ANALOGTRIGGER` **is derived from the digitizer's sample stream, so the comparator only evaluates while a measure/digitize block is running.** With the wait first, nothing is sampling, the comparator never sees the signal, and the wait cannot be satisfied. There is no diagnostic — the model simply sits, which is indistinguishable from a trigger that has not arrived yet.

This is a trap worth stating plainly because the broken program looks *better* than the working one: it is two blocks instead of four and it is on the fast path (§10). It is on the fast path because it never does anything.

`EVENT_EXTERNAL` **as the first block works, so the restriction really is the comparator's** (MEASURED, P6, rear TRIGGER IN driven by a 100 Hz pulse). A two-block program — `WAIT EVENT_EXTERNAL` then a counted digitize — completed inside the first second with its full 100 readings. The control is the same program with the marker switched off: it sat the whole four-second window at **zero readings**, so nothing but the marker ended it.

A FINISHED COUNTED PRE-ROLL DOES SATISFY THE COMPARATOR WAIT — BECAUSE OF THE LATCH, NOT THE COMPARATOR (MEASURED, same). 100 samples at 40 kS/s, finished, then `WAIT EVENT_ANALOGTRIGGER`: `IDLE` with 200 readings, apparently answering the question with a yes. **It is the wrong yes.** Those 100 samples span 2.5 ms of a 1 kHz square, so an edge crossed the level *while the block ran*, and the default `CLEAR_NEVER` let the wait consume what that edge latched. The same program with `trigger.CLEAR_ENTER` on the wait sat the full window at exactly **100 readings** — the pre-roll's own samples and not one more. So the comparator does **not** evaluate once the measure block has stopped, and what a finished pre-roll leaves behind is a pending event rather than a live comparator. Both readings of "it fired" are true and only one of them is a fact about the comparator.

NOT ESTABLISHED: whether a wait on `EVENT_DISPLAY` as the first block works. It needs the front-panel TRIGGER key, which is the one stimulus here with no programmatic source — the LXI virtual panel can press it over HTTP, and that path is closed by policy rather than by the instrument.

The wait also keeps the pre-roll running, and does not only await the trigger (MEASURED, P2f). An infinite digitize as the *only* block idles after about a dozen samples whatever `dmm.digitize.count` says, because the sequence ends and that stops the block. With the wait downstream the same block filled a 20 000-reading buffer and sat in `WAITING`. The counted block stops the infinite one; the wait is what lets it run at all.

The window therefore contains everything from `initiate()` onward, so a capture armed this way cannot miss the start of the signal it waits for. Pre-trigger depth is bounded by buffer capacity, not by a percentage.

The comparator's level and slope are both honoured (MEASURED, P2, against a 1 kHz 0-3 V square wave). Armed at 8 V — outside the signal's range — the model sat in `STATE_WAITING` indefinitely and never fired, so the level is real and not a formality. Armed at mid-swing, reading either side of the trigger index: `SLOPE_RISING` gave −0.001 V before and 3.000 V after, `SLOPE_FALLING` gave 3.000 V before and −0.001 V after, each with a single partial-transition sample at the edge. Opposite directions from the two settings.

An explicit count on the block **overrides `dmm.digitize.count`** (MEASURED, E3e: count 7 set, block count 250, 250 readings taken). Counts of 8192, 32768 and 1 000 000 are all accepted (MEASURED, E3f).

The reserve is the wire BEFORE the device started, and it out-measures everything real in the record

A window that holds everything from `initiate()` onward is also a hazard. On an arm from silence the reserve is not a quiet stretch of the signal: it is a measurement of a switched-off wire, taken before the device under test transmitted at all. Any reader that infers a property of the signal from the record's longest

or leading stretch — a polarity, an idle level, a baseline, a noise floor — will take it from the reserve and be **self-consistently wrong**: the inference succeeds, reports a plausible quality figure, and is wrong in every sample it touches.

Its length disarms the usual protection. The natural guard against a bad inference is a plausibility test — *is this stretch long enough to be a real idle?* A reserve of a thousand samples clears any such threshold several times over, so the capture that most needs the sanity check is the one certain to pass it. A guard of that shape protects every capture except this one.

It is a sample-rate boundary, not a coin toss. The reserve is a fixed SAMPLE count — a percentage of buffer capacity, 1055 samples at 5 % of a 21 100-reading buffer — while a gap in the signal is a fixed TIME. So whether anything real out-measures the reserve is decided entirely by the sample rate. MEASURED, armed on a silent line with the source switched on at +3 s, six captures a condition:

condition	leading run	next run	the next run in TIME
busy line, never armed	50	344	—
armed, 200 kS/s	1052	1210	6.05 ms
armed, 80 kS/s	1053	500	6.25 ms

The second run is the **same 6 ms gap** in both armed rows, while the leading run is the reserve at its fixed ~1055 either way. So the reserve dominates at the lower rate and is overtaken at the higher one with nothing else changed — which is why a defect of this kind reproduces at one sample rate, vanishes at another, and looks intermittent until the arithmetic is written down. For a 6 ms gap the crossover is about **171 kS/s**.

Both polarities are affected, in opposite directions. On a single-supply line the dead wire sits at ground, which is also the signal's low level, so the reserve reads as a long low. On a ground-straddling line the threshold sits near 0 V and the dead wire sits all but *on* it, so which side the reserve falls on is decided by millivolts: MEASURED at a −0.011 V threshold, the dead line landed on the HIGH side by 11 mV.

The remedy is to tell the reader that the record opens before its own trigger, and the flag has to be conditional on the reserve existing: at a pre-trigger of **0** the record opens ON the edge and its leading stretch really is the signal, so a flag set unconditionally is a constant and protects nothing. Exclude the leading run from any inference about the signal, publish it as its own quantity, and let the interior of the record decide. Where the window holds nothing but the reserve, the plausibility test fires honestly and whatever prior the signalling standard supplies is used instead.

With the leading run excluded, every armed condition above recovered to the same success rate as the never-armed control, and on a ground-straddling line 36 of 36 captures across three bands and two configurations read the polarity correctly.

And assert the premise, not only the outcome. A test that checks the end result is **vacuous when there was no reserve to begin with** — it passes for the wrong reason. The assertion worth writing is that the arm fired *and* the leading run exceeded a few hundred samples. MEASURED with the remedy disabled: the premise check fails in every configuration while the end-to-end check fails in only one of three, because a downstream contest rescued the other two on that run. The premise check is the reliable detector; the outcome check is what says the defect reached a user. Keep both.

8. The canned templates, disassembled

`LoopUntilEvent(event, position, clear, delay, buffer)` — the shape above (MEASURED, E1a):

```
1) BUFFER_CLEAR  2) DELAY_CONSTANT 0  3) MEASURE_DIGITIZE INFINITE
4) WAIT <event> CLEAR:ENTER           5) MEASURE_DIGITIZE <count>  6) NOTIFY 2
```

`position` is a pre-trigger **percentage**, and the post-trigger count the template programs is

```
count = (100 − position)% of the BUFFER'S CAPACITY
```

`dmm.digitize.count` has no part in it. MEASURED, E4: capacity and position varied independently across 15 combinations, each matching the rule exactly, and `dmm.digitize.count` set to 77 against 1000 changed nothing.

capacity	position 0	5	20	50
10 000	10 000	9 500	8 000	5 000
20 000	20 000	**19 000**	16 000	10 000
50 000	50 000	47 500	40 000	25 000

So an armed capture never fills its buffer, and the shortfall is not a fault. At `position` 5 the template programs 95 % of capacity as the post-trigger count, and the readings above that come from the pre-trigger phase — however many accumulated between `initiate()` and the trigger arriving. On a continuously active line that is a handful of samples, not the 5 % the position nominally reserves, so a completed capture lands a little over 95 % of capacity and never at 100 %. Testing for a full buffer as the completion condition therefore fails on every capture that worked; `trig_done()` is the test that holds.

Reaching the wait block takes about 0.19 s, and it does not depend on the buffer. MEASURED from inside the instrument, 30 arms at each of two capacities, polling with `delay(0.001)`:

capacity	min	median	p95	max
21 100	0.1668	**0.1917**	0.1932	0.2032
860 000	0.0210	**0.1928**	0.1952	0.1973

A sweep of seven capacities from 21 100 to **2 800 000** — a 133-fold range — is flat at 0.171 to 0.180 s. **Five controls are nulls**: `dmm.digitize.count` (1 against 817 000), a host `buf.clear()` before `initiate()`, `samplerate` (40 kS/s against 1 MS/s), template `position` (1 against 50), and the poll cadence itself (10 ms, 1 ms and 0.2 ms alike).

What does move it is **what the instrument was doing before the arm**:

the previous model	time to `STATE_WAITING`
aborted while sitting in `STATE_WAITING`	**0.228 s** — the worst case
a `SimpleLoop` that ran to `IDLE` on its own	0.177 s
a completed `dmm.digitize.read()`	0.164 s
`load("Empty")`, nothing acquiring	0.191 s

And a caller that aborts before every capture always pays the worst row. Aborting for hygiene makes the prior state permanently "aborted while waiting" — 0.228 s, the slowest row in the table — so the tidiest caller is the one with the least margin. A spin bounded at 0.25 s took its last look at about **0.232 s** (25 iterations at a measured 9.67 ms each): **4 ms of margin**, at every buffer size.

The `delay()` in the spin is load-bearing. Cadence between 10 ms, 1 ms and 0.2 ms is a null, but **zero** delay is fatal: a tight loop issued **56 102** `pcall(trigger.model.state)` calls over 3 s and the model sat at block 2 — the zero `DELAY_CONSTANT` — the whole time, never arming. The state read is IPC to the second processor, and a spin that never yields starves the trigger engine. Deleting the `delay()` as a micro-optimisation would break every armed recording.

So a bound anywhere near 0.2 s is a coin toss. A spin bounded at 0.25 s sits about ten per cent above the slowest arm the instrument produces, and a test of it armed once and then failed three consecutive runs. A bound of **10 s** — forty-four times the slowest arm — is not generous: the distribution has a hard floor at 0.164 s and the cost of overshooting the bound is total, so there is nothing to buy by trimming it.

The consequence inverts the feature instead of degrading it. A spin that never sees `STATE_WAITING` has to report an arm that did not go live, because the poll loop cannot tell that from a trigger that fired — so the recording runs free and captures exactly the silence that arming exists to avoid.

And the 0.19 s is a reporting lag, not an arming delay. The wait block goes live about **2 ms** after `initiate()`; the state call simply does not admit it for another 190. Measured two independent ways that agree. From the recorded data, `pre-roll start = fire_s − N5/fs − n_pre/fs` over six captures at the two rates where the post-trigger block dominates the elapsed time: **1.6 to 2.6 ms**. And from the block trajectory, `BUFFER_CLEAR` completes at 0.0003 s and the zero `DELAY_CONSTANT` at 0.0012 s. `CLEAR_ENTER` discards any crossing latched before the block was entered, so the crossing that fired is genuinely after entry — which means the wait was entered by then.

In twelve latency captures `STATE_WAITING` was **never once observed**, because the signal fired within one ramp period of the wait going live, long before the state reported it.

So a capture that fails a short spin was armed the whole time. It is declared unarmed on the strength of a stale state read, and no data was ever lost to a late comparator. A generous bound is still the right fix — polling the state is the only way to ask — but the number bounds the firmware's candour, not its readiness.

The comparator fires about 7.3 µs after the crossing, and that is fixed in time

MEASURED against a slow ramp, so the crossing can be located by interpolation rather than quantised to a sample. The latency is the distance between the interpolated crossing and the trigger point the pre-trigger reserve puts in the record:

`samplerate`	slew	latency, samples	latency, µs	µs less one sample
2 500	6.25 V/s	1.308	523.34	123.33
40 000	100 V/s	1.380	34.49	**9.49**
200 000	500 V/s	2.432	12.16	**7.16**
1 000 000	2 500 V/s	8.361	8.36	**7.36**

Fixed in TIME, not in samples. Between 200 kS/s and 1 MS/s the rate changes 5.0×, the latency in samples changes 3.44×, and the latency in microseconds changes **1.03×**.

Two honest caveats. The trigger instant is attributed to the first post-trigger sample; attribute it to the last pre-trigger sample instead and every figure drops by exactly one sample period, which is the right-hand column — a definition, not something the data can settle. And the **2 500 S/s row cannot resolve this at all**: one sample is 400 µs there, so the ±0.5-sample ambiguity is ±200 µs, thirty times the quantity being measured. **Jitter is comparable to the latency** — 6.2 to 9.3 µs at 1 MS/s, 3.6 to 9.9 µs at 200 kS/s — so the firing point is not deterministic to the sample.

`buf.n` is not a clock above 100 kS/s, but the sample rate is real. The ramp slope measures the true sample interval directly and recovers **1 000 673 to 1 000 996 S/s** when 1 MS/s is asked, and 200 127 to 200 193 when 200 kS/s is asked. What lags behind the acquisition is `buf.n`, which is what the `2872` / `2873` "Begin/End reading backlog processing" pair reports.

`buffer.make()` is cheap and is not in that interval. It measures 2.5 to 3.7 ms at 860 000 readings over 21 observations, and it runs before the model is loaded, so no part of allocation is spent between `initiate()` and the wait. Two readings of 0.78 s appear in one sweep and are outliers: the other nineteen observations at the same capacities refute a per-reading cost, and a linear model built on them over-predicts the 860 000 case by a factor of seventy.

`SimpleLoop(count, delay, buffer)` — a counted loop with **no wait at all** (MEASURED, E1b):

```
1) BUFFER_CLEAR  2) DELAY_CONSTANT 0  3) MEASURE_DIGITIZE <buf>, 1
4) NOTIFY 2      5) BRANCH_COUNTER <count> → 2
```

`SimpleLoop` acquires the instant it is initiated. A recording built on it **cannot** be armed, and a 32 768-sample recording executes 32 768 iterations of a three-block loop.

9. A worked example: the firmware's own scan program

The best teacher is the program the instrument writes for itself. A 6-channel RTD scan — `scan.add("1,6,2,3,4,5")`, `scancount` 10000, `scaninterval` 10 s, `scan.export(..., WRITE_AT_END)`, on a `2000,10-Chan Mux` — generates **29 blocks in two sequences** (MEASURED):

```
A  1) START
   2) CONFIG_RECALL  ChannelList INDEX 1      -- the INIT step: open all
   3) CHANNEL_ACTION
   4) WAIT  EVENT_TIMER4   CLEAR:NEVER        -- the interval gate
   5) NOTIFY 5                                -- restarts timer 4
   6) CONFIG_RECALL  ChannelList INDEX 3
   7) BRANCH_ONCE   → 11                      -- cold start: no prior measurement to
wait for
   8) BRANCH_ALWAYS → 10
   9) CONFIG_NEXT    ChannelList
  10) WAIT  EVENT_NOTIFY7  CLEAR:NEVER        -- "the measurement finished"
  11) CHANNEL_ACTION                          -- close the next channel
  12) NOTIFY 6                                -- "the channel is ready"
  13) BRANCH_COUNTER 6     → 9                -- inner loop: 6 channels
  14) BRANCH_COUNTER 10000 → 4                -- outer loop: 10000 sweeps
  15) END

B 16) START
  17) BUFFER_CLEAR   defbuffer1
  18) WAIT  EVENT_TIMER4   CLEAR:NEVER        -- the same interval gate
  19) CONFIG_RECALL  MeasureList INDEX 3
  20) BRANCH_ALWAYS → 22
  21) CONFIG_NEXT    MeasureList
  22) WAIT  EVENT_NOTIFY6  LOGIC:AND          -- "the channel is ready"
  23) MEASURE_DIGITIZE  defbuffer1  COUNT:AUTO
  24) NOTIFY 7                                -- "the measurement finished"
  25) BRANCH_COUNTER 6     → 21
  26) NOTIFY 8                                -- "the sweep finished"
  27) BRANCH_COUNTER 10000 → 18
  28) WRITE_USB      defbuffer1               -- scan.export, WRITE_AT_END
  29) END
```

Five idioms worth stealing, none of them in the reference manual:

A ping-pong rendezvous. A closes a relay and raises `NOTIFY 6`; B waits on it, measures, raises `NOTIFY 7`; A waits on that before advancing. Neither sequence can outrun the other. `CLEAR:NEVER` is deliberate here and is what makes a notify behave as a semaphore rather than a race — a notify raised before the partner reached its wait is still honoured.

A self-restarting interval timer. `trigger.timer[4]` has `delay = 10`, `count = 1` and `start.stimulus = trigger.EVENT_NOTIFY5`. Blocks 4 and 18 wait on `EVENT_TIMER4`; block 5 raises `NOTIFY 5`, which starts the timer. Each sweep therefore re-arms the clock at its own start, giving a period measured sweep-start to sweep-start rather than 10 s of dead time added to a sweep of unknown length. The timer configuration lives outside the trigger model, in the setup script.

Loop peeling with `BRANCH_ONCE`. Block 7 skips the `WAIT NOTIFY7` exactly once, because on the first channel of the first sweep there is no earlier measurement to wait for. Every later pass falls through to block 8 and waits properly. One block replaces a prologue.

Configuration lists as the data plane. Per-channel settings are not expressed as branches. Two lists — `ScanAutoCreatedChannelList` and `ScanAutoCreatedMeasureList` — are stepped by `CONFIG_NEXT` inside each inner loop, and `MEASURE_DIGITIZE` uses `COUNT:AUTO` to take each channel's own count from the recalled configuration. The program is the control flow; the lists are the parameters.

No blenders. Both are untouched — `orenable` false, every stimulus `EVENT_NONE`. Keithley synchronises with notifies and timers alone. For a simple OR of two trigger sources a multi-event `WAIT_OR` is the idiomatic answer, and it costs no blender at all.

Two cautions this example also demonstrates. `WRITE_AT_END` puts the USB write in block 28, so a power loss anywhere in the preceding 27.8 hours costs the entire record. And `scan.monitor.channel` set without `scan.monitor.mode` generates **no** limit or branch-on-limit blocks at all: the monitor is inert and the scan simply runs on its interval.

The first sweep stalls for one whole interval, and the two steps that establish it are worth keeping because the conclusion is not obvious from the listing. A timer carrying a `start.stimulus` does **not** start when it is enabled: MEASURED (A13) — `timer[1]` with `delay = 1.0`, `start.stimulus = EVENT_NOTIFY5` and `enable = ON`, with a program waiting on `EVENT_TIMER1`, sat in `STATE_WAITING` with 0 readings three seconds later. Nothing therefore raises `EVENT_TIMER4` until block 5 runs, and block 5 is downstream of the wait at block 4. `CLEAR:NEVER` cannot rescue it either, because a latch needs an expiry to latch and there has been none.

So the scan takes `scaninterval` seconds to produce its first reading. On a 27.8-hour run that is invisible, which is presumably why it has never been worth fixing.

10. Performance, and the oracle the firmware gives you

`getblocklist()` ends with `* This trigger model may not be set up for low latency. *` whenever the program is off the fast path. That makes the boundary observable, and it is **a list of specific disqualifiers, not a size heuristic** (MEASURED, verify_latency).

What costs the fast path:

disqualifier	measured boundary
measure/digitize blocks	1 and 2 keep it; **3 loses it**
total blocks	14 keeps it; **15 loses it**
`BUFFER_CLEAR`	**any**, even one
`DELAY_CONSTANT`	0, 0.1 and 0.25 s keep it; **0.3 s loses it** — the boundary lies in 250-300 ms
`LOG_EVENT`	**any**, with a valid event number
two blocks waiting on the **same** event	**any** — see below
a configuration-list block (`CONFIG_RECALL` / `NEXT` / `PREV`)	**any**, even one
a branch to a block that **does not exist**	**any**

And being off it costs nothing measurable. The flag is an oracle with no units attached, so the penalty was measured directly, against the comparator latency, on three program shapes:

shape	blocks	low-latency warning	latency at 1 MS/s
A the canned `LoopUntilEvent`	6	**present**	8.23 ± 1.26 µs (n=20)
B hand-built 3-block equivalent	3	absent	8.05 ± 0.98 µs (n=20)
C A with ONLY the `BUFFER_CLEAR` removed	5	absent	8.00 ± 1.00 µs (n=18)

B − A = −0.34 ± 0.32 µs and C − A = −0.49 ± 0.34 µs, inverse-variance weighted over the two rates that resolve microseconds. Both are 1.1–1.4 σ, and the 95 % upper bound on the benefit is **1.2 µs** against a total comparator latency of about 8 µs. Fast-path status was read from the firmware's own warning string, not inferred from the table above.

C is a result in its own right: `BUFFER_CLEAR` is the SOLE disqualifier in the canned template. Removing it alone clears the warning, so the zero `DELAY_CONSTANT` and the `NOTIFY` cost nothing.

Throughput is a null too, and no sample is dropped at the trigger. The fitted per-sample step of a ramp is the dropped-sample detector: **0.002498 V against 0.002500 programmed** for all three shapes at both rates, and all three deliver the full programmed 20 045 post-trigger readings at 1 MS/s. So the handover from the infinite pre-trigger digitize to the counted post-trigger one loses nothing — acquisition is continuous across the trigger, which is what makes the pre-trigger reserve possible.

One difference that could not be removed, and it is inert rather than fixed: A's wait reports `LOGIC: OR` while both hand-built waits report `AND`, because `setblock` cannot take a logic argument without a second event. With one event the two mean the same thing.

So there is no latency and no throughput case for restructuring a four-block armed capture, and §8's control agrees from the other side — moving the clear out of the model does not shorten the arm either, 0.1763 s against 0.1788 s.

What is free: `NOP`, `NOTIFY`, `WAIT` (including a 3-event `WAIT_OR`), `BRANCH_COUNTER`, `BRANCH_ALWAYS` **to a target that exists**, `COUNT_INFINITE`, a second measure block writing to a **different buffer**, and **any number of `DELAY_CONSTANT` blocks** — six were free (MEASURED, P4), so the delay disqualifier is duration and not count.

The firmware states the measure-block limit itself. Building a third measure block posts

```
2753: Trigger model is not optimized for speed due to greater than 2 measure/digitize
blocks
```

which is a primary source for the boundary and better evidence than the sweep that found it.

TWO WAITS ON ONE EVENT IS A REAL DISQUALIFIER, and it is the row most easily mistaken for something else (MEASURED, P1e). Holding the program shape fixed and varying only the events:

```
one WAIT on EVENT_DISPLAY, then a measure          LOW LATENCY
two WAITs, both on EVENT_DISPLAY                   warned
two WAITs, on EVENT_DISPLAY and EVENT_EXTERNAL     LOW LATENCY
```

It holds at two different block counts, so the cost is the shared event and not the extra blocks. This is also why a `WAIT / MEASURE / WAIT / MEASURE` program warns while `WAIT` itself is free — an earlier reading of that program wrongly suspected `WAIT`.

`BRANCH_ALWAYS` costs nothing; a dangling target costs everything (MEASURED, P1a). A `BRANCH_ALWAYS` to a block that exists is free, backwards or forwards; the same opcode pointing past the end of the program warns. So the disqualifier is the unresolvable target, not the jump.

A REJECTED BLOCK READS AS A FREE ONE, and that is a trap in measuring any of this. `LOG_EVENT` first measured as free because the probe passed a bare `1` as the event number: `setblock` returned true, the block was never added, and a program of one measure block is of course on the fast path. The refusal

was event **2725**, "Log Event block number 2 Event Number is invalid", visible only in the log. With `trigger.LOG_INF01` (2734) the block is accepted and the warning appears. **Confirm a block is present in `getblocklist()` before drawing any conclusion from its absence of effect** — see §5.

The manual never uses the words "low latency" — zero occurrences in 55 488 lines. The firmware's vocabulary for its own fast path is undocumented, and the manual nowhere connects the flag to its performance advice. That advice is a list of SEVEN bullets at §9-50, headed "Improving the performance of a trigger model". Four of them turn out to be exactly what the flag enforces and three are wrong as written:

§9-50 says	measured
fewer than 15 blocks	**agrees** — 14 keeps the flag, 15 loses it
constant delays under 254 ms	**agrees** — 0.25 s keeps it, 0.3 s loses it
four or fewer measure/digitize blocks	**stricter than documented** — only 2 keep it
do not use multiple reading buffers	**contradicted** — a second buffer costs nothing
four or fewer delay blocks	**not enforced** — six were free
no two blocks waiting on the same event	**agrees** — and it is enforced, not advice
limit configuration-list blocks	**agrees, absolutely** — one block is enough to lose the flag

The practical consequence is the opposite of what it looks like. The two armed shapes most worth building are both on the fast path:

```
MEASURE_DIGITIZE INFINITE / WAIT / MEASURE_DIGITIZE n     LOW LATENCY   (pre-trigger)
WAIT / MEASURE_DIGITIZE n                                 LOW LATENCY   (armed
recording)
```

while `LoopUntilEvent` and `SimpleLoop` are **not** — and the only reason is the `BUFFER_CLEAR` block each template emits. Clearing the buffer from Lua with `buf.clear()` before `initiate()` instead of with a block keeps an otherwise identical program on the fast path. Every scan is off it too; the 29-block example above exceeds the 14-block limit on its own.

The disqualifiers have a shape, though a weaker one than it first appears. `BUFFER_CLEAR` is a side-effecting operation, a long delay blocks, a dangling branch target cannot be resolved, and two blocks waiting on one event cannot be ordered without knowing which consumes it. A plain unconditional jump, a counted branch and a wait are all free. That is consistent with a fast path that resolves a bounded program ahead of time and falls back to general interpretation when it cannot — but **the mechanism is not stated anywhere in the manual and this document does not claim to have established it.** An earlier draft offered "`BRANCH_ALWAYS` is an unconditional jump" as evidence for that shape; measurement refuted it, and the example is gone.

11. Documentation defects found

- **Notify count, given inconsistently nine times against two.** The event tables say "Notify trigger block N (**1 to 3**)" at ref lines 13930, 13962, 18230, 18646, 19044, 25468, 26845, 27086 and 27417, and "1 to 8" at only 25774 and 26607. The wrong value is the one that dominates. **Eight is right**: `trigger.EVENT_NOTIFY8` resolves on the instrument, and the scan generator emits `NOTIFY 6`, `7` and `8`.

- **Out-of-band failure reporting is undocumented.** Nothing states that `setblock` and `initiate` report refusals as event-log entries while returning success, and no example checks for it.
- **A constant that does not exist.** The SCPI mapping table at ref line 13860 gives the paused state as `trigger.STATE_PAUSE`. That constant is **nil** on the instrument; the real one is `trigger.STATE_PAUSED`, as ref line 49484 has it. And that state is reachable: `pause()` on a model that **is** running returns true and the state becomes `trigger.STATE_PAUSED`, with `resume()` putting it back to `RUNNING` (MEASURED, P6).
- **A second nil constant, and it fails as a trigger-model error.** `buffer.STYLE_COMPACT` is **nil** on 1.7.17a, so `buffer.make(n, buffer.STYLE_COMPACT)` hands nil to the style parameter and posts **1138**, "Parameter 2, expected type 'rbuffer_style', but found type 'nil'". The cost is not one event: every `setblock` built on the nil result posts **2703**, the program reaches `STATE_FAILED`, and `pcall` returned true throughout — so the symptom presents as a broken trigger model and the cause is a buffer constant four events earlier in the log. `buffer.STYLE_STANDARD` resolves.
- **Four undocumented block types.** `START`, `END` and `CHANNEL_ACTION` exist as TSP constants and appear in no `setblock` usage list. `WRITE_USB` appears in the manual not at all.
- **An exhausted branch counter reads 0, not one past its target.** Ref line 54192 has it that "when the trigger model reaches the set counter value, branching stops and the counter value is one greater than the setting", so a counter set to 20 should read 21 afterwards. It reads **0** (MEASURED, G5). One model, one statement, two counters: the free-running one read 20 while the exhausted one read 0, and the same pair read 11 and 10 mid-run, so both were counting normally until one hit its target. Reset at the end of the run is separately ruled out, because the free-running counter was read after the model reached `IDLE` and still held its count. The firmware self-resets a counter on exhaustion, which is what a nested inner loop needs and what the documented value would break. The consequence for a probe: a counter read back as a program counter must be capped beyond reach, or it reports zero in exactly the case where it ran every lap.

12. Choosing a program

To capture on a signal edge and keep what preceded it, use §7's shape: an infinite digitize, a wait, then a counted burst. The window then contains everything from `initiate()` onward, so the start of the signal cannot be missed. `LoopUntilEvent` is the canned equivalent and costs the fast path.

To arm a long acquisition on the analog trigger, use §7's four-block shape. **Do NOT reduce it to a wait followed by a counted `MEASURE_DIGITIZE`** — that program builds cleanly, passes every static check, sits on the fast path, and never fires. See the warning below.

To read what comes back, do not treat the record's leading run as a measurement of the line. It is the reserve, and on an arm from silence it is the wire before the device started — see §7.

To clear the buffer, call `buf.clear()` from Lua before `initiate()` rather than using a `BLOCK_BUFFER_CLEAR`. The block is a fast-path disqualifier and the Lua call is not.

To OR two or three trigger sources, put them in one `WAIT` with `trigger.WAIT_OR` instead of configuring a blender. Fewer blocks, no shared subsystem to release afterwards.

For anything a human can trigger, use `trigger.CLEAR_ENTER`. The default `CLEAR_NEVER` honours a press made before the block was reached.

To record a bursty line and discard the silence between bursts, use §14's shape: §7's four blocks with a counter behind them and two buffers, the gap aimed at a scratch bin. Detect the end of each burst with §13's construct rather than a fixed burst length — a fixed length splices inside traffic and loses the frame

that straddles the seam.

Read `trigger.model.state()` after every build and `initiate()`, and drain `eventlog.next()`. Neither call reports its own refusal.

13. Detecting silence: `BRANCH_ON_EVENT` consumes its event

There is no trigger event for a line going quiet. The comparator fires on a crossing and silence raises nothing, so no `WAIT` can be armed on it. The *absence* of a crossing over a window is still expressible, because `BRANCH_ON_EVENT` tests an event rather than blocking on it:

```
k)   DELAY_CONSTANT      0.1
k+1) BRANCH_ON_EVENT     <event>, k      a crossing arrived: go round
k+2) ...                                 fell through: quiet for 100 ms
```

This works only if re-entering block k+1 branches again on an event that arrived *since the last entry*, and nothing published settles that. `BLOCK_WAIT` takes a `clear` argument precisely because it has a latch-consumption policy; `BRANCH_ON_EVENT` takes none, so the behaviour is not selectable. §6 records the latch clearing at the start block and on exiting a wait, and is silent about a branch. The reference leans one way without saying it — "after the trigger event occurs, the next time the trigger model reaches the branch-on-event block, it goes to the branching block" (14-381) — but "the next time" says nothing about the time after that.

The branch consumes the event, and the detector re-arms once per event (MEASURED, G1–G3). Three stimuli through one construct, with a `BRANCH_COUNTER` capped beyond reach at each landing site as a program counter:

event	stimulus	laps	branches	reading
`EVENT_NOTIFY1`	one notify block	2	**1**	consumed; fell through on re-entry
`EVENT_TIMER1`	0.5 s period	100	**20**	one branch per expiry, not per lap
`EVENT_ANALOGTRIGGER`	2 Hz square	100	**20**	one branch per crossing
`EVENT_ANALOGTRIGGER`	1 kHz square	100	**100**	every window busy; the loop never exits
`EVENT_ANALOGTRIGGER`	DC, no crossing	**1**	0	exits one window after silence

The notify row alone would not have settled it. A detector consumed once and never re-armable predicts that same single branch and would be just as useless, so the periodic rows are what show the re-arm. Their counts are predictions rather than inequalities: 100 laps of 0.1 s against a 0.5 s period is 20 events, and that is what came back on both the timer and the comparator.

The comparator keeps posting while the loop runs. §7 establishes that `EVENT_ANALOGTRIGGER` is derived from the digitizer's sample stream, so the comparator evaluates only while a measure block runs — which raises the question of whether it survives a loop made of delay and branch blocks. It does, provided a `COUNT_INFINITE` measure block is still live; the comparator rows above were taken with exactly that shape and the DC row is their control. The quiet row exited after **1001 samples at 10 kS/s, 100.1 ms**: one window and no more.

Two limits worth stating. The construct's resolution is its `DELAY_CONSTANT`, so a 0.1 s window cannot tell 0.1 s of quiet from 0.19 s. And the window is a *minimum* observable gap, not a latency — a burst is detected as over no sooner than one window after its last edge.

14. A re-entered measure block appends, and the timestamps carry the splice

§7's four-block shape records one burst. Putting a counter behind it records many bursts into one buffer while the silence between them goes somewhere else:

```
1) BUFFER_CLEAR      rec                       once, outside the loop
2) MEASURE_DIGITIZE  scr, COUNT_INFINITE       the gap's samples, discarded
3) WAIT              EVENT_ANALOGTRIGGER, CLEAR_ENTER
4) MEASURE_DIGITIZE  rec, 500                  the burst
5) BRANCH_COUNTER    4, 2
6) DELAY_CONSTANT    0
```

`rec` **accumulates across passes** (MEASURED, G4). Four passes of 500 readings gave `rec.n` = **2000**, not 500, so block 4 appends rather than restarting its record. E3d had measured accumulation across one wait; this is the second pass and later. `startindex` stayed 1, so a record that never fills is not rotated. The silence went where it was aimed: `scr.n` = 13 231, which is 1.3 s discarded against 0.2 s kept.

`trigger.CLEAR_ENTER` on the wait is load-bearing rather than cautious. The default `CLEAR_NEVER` lets the wait consume a crossing latched during the burst it has just recorded, so it falls straight through, the next pass starts immediately, and no gap is ever skipped. The program still runs and still fills its record; the elision simply does nothing.

The span-derived sample rate is wrong by the elision ratio. `(n−1)/span` over `buf.relativetimestamps` is how `acq_measure_fs` in `tsp/serial_core.tsp` reads the rate back, and it is the number every bit time is scaled by. Elision removes the gap's samples and not the gap's time, so on the record above the span was 1.5737 s for 2000 readings: **1270 S/s implied against 10 000 commanded, low by 7.9x**. Any decode driven off the whole-span average is scaled by that factor.

The splices are exactly recoverable, which makes it repairable (MEASURED, G4). Every reading keeps its own timestamp, so a gap appears as a discontinuity between consecutive ones. On the same record: **3 splices, at indices 501, 1001 and 1501** — the four-burst boundaries exactly — with deltas of 0.4743, 0.4497 and 0.4500 s against a commanded 0.5 s generator period. All 1996 remaining deltas were **100 µs**, giving 10 000 S/s inside the bursts and a ratio of **1.000** against the commanded rate. So segment the record at the discontinuities and measure fs within a segment: only the whole-span average is corrupt, and the record carries everything needed to avoid it.